

Ontology-Grounded Retrieval for Auditable LLM-Assisted Secure Code Generation

Pedro Farinha

Independent Researcher

pedro.farinha@shiftleft.pt

github.com/pedrofarinhaatshiftleftpt

Supported by Shiftleft - Secure Software Engineering, lda.

Abstract

Large language models (LLMs) generate code with security vulnerabilities at significant rates [1, 2, 3]. Recent work shows that security-focused prompting can reduce vulnerability rates [4], suggesting that LLMs can produce more secure code *when given structured security context* — but current retrieval approaches lack the requirement-level grounding needed to do this systematically.

We propose ontology-grounded retrieval, a method for providing LLMs with typed, weighted, and provenanced security knowledge drawn from a normalized domain ontology. Unlike plain vector-similarity RAG — which retrieves relevant text without provenance, epistemic weight, or completeness guarantees — ontology-grounded retrieval operates through structured queries against a typed knowledge graph, producing a retrieval result with six formally distinguishable properties: first-class provenance, explicit typing, epistemic weighting, reproducible grounding, assessable completeness, and verifiability against the same index.

We define a retrieval contract with completeness and provenance invariants, present a verification taxonomy (syntactic, semantic, human) that makes automation boundaries explicit, and illustrate the method through three compact scenarios. The validation is methodological, not experimental; a controlled evaluation is identified as future work.

The method does not make LLM output secure. It makes the grounding auditable: for any generated code, one can determine which security requirements informed it, at what authority level, and whether all applicable requirements were addressed.

Keywords: LLM, code generation, application security, ontology, retrieval-augmented generation, structured retrieval, requirements traceability, verification

1. Introduction

LLM-assisted code generation has achieved wide adoption [5], but the security properties of generated code remain uncontrolled. Evaluations across multiple models and languages show vulnerability rates of 25–40% [1, 2, 3], spanning injection, authentication, cryptographic misuse, and configuration errors.

A natural mitigation is to inject security documentation into the LLM’s context via retrieval-augmented generation (RAG) [6]. If the LLM has access to relevant security guidelines, it should produce more secure code. In practice, however, this approach provides security context through vector similarity search — a mechanism that is probabilistic, untyped, and provenance-free. The LLM receives text that is similar to the query but has no structured understanding of:

- Which requirement the text represents

- At what authority level — mandatory requirement or informational example?
- Whether all relevant requirements were retrieved — similarity has no concept of domain coverage
- How to verify that the output satisfies the retrieved requirements

This paper proposes ontology-grounded retrieval as a methodological alternative. The method uses a normalized security ontology as the retrieval substrate, producing typed, weighted, provenanced context that enables a closed verification loop. The method is general: it requires a typed ontology with explicit relations, a stratified structural index, and a delivery protocol. We instantiate it using AppSec Core [7] (a normalization ontology for application security) and the Model Context Protocol (MCP) [8], but the method is not bound to these specific components.

Research Question

How can structured, ontology-based retrieval improve reproducibility, completeness, and traceability in LLM-assisted secure code generation — compared to plain vector-similarity retrieval?

Contributions

1. Ontology-grounded retrieval as a method: a five-stage pipeline (intent parsing → structured retrieval → context assembly → grounded generation → verification) with formally distinguishable properties.
2. A retrieval contract with completeness and provenance invariants that make grounding testable.
3. A six-property comparison between plain vector-similarity RAG and ontology-grounded retrieval (provenance, typing, weighting, reproducibility, completeness, verifiability).
4. A three-class verification taxonomy (syntactic, semantic approximation, human judgment) that makes automation boundaries explicit.
5. Three compact illustrative scenarios showing how the audit trail is intended to operate from prompt through verification.

Scope

This paper describes a method, its contract properties, and its AppSec-specific instantiation. We do not claim empirical superiority over alternative retrieval architectures; we claim that the proposed structure makes grounding auditable and its completeness properties assessable. The method is general in structure but validated here only in an AppSec instantiation using AppSec Core [7] and MCP [8]. Generality beyond this domain is argued, not demonstrated. A controlled evaluation is future work.

2. Background and Related Work

2.1 LLM Code Generation and Security

Modern LLMs generate syntactically and often semantically correct code [5], but optimize for functional correctness, not security. Pearce et al. [1] found that ~40% of Copilot-generated programs contained vulnerabilities. Tihanyi et al. [2] confirmed this at scale across 9 LLMs and 331,000 programs. Tony et al. [3] developed LLMSecEval as a standardized security benchmark.

Tony et al. [4] demonstrated that security-focused prompting (recursive criticism, chain-of-thought with security examples) can reduce vulnerability rates by up to 75% on some tasks. This is a key observation: LLMs can produce more secure code when given appropriate security context. The question is how to provide that context systematically, completely, and auditably.

2.2 Retrieval-Augmented Generation

Lewis et al. [6] introduced RAG for grounding LLM outputs in external knowledge. RAG is a family of architectures; our critique targets specifically *plain vector-similarity retrieval without structured metadata*. Hybrid systems with metadata filtering, re-ranking, or graph-based retrieval [9] address some limitations. Ontology-grounded retrieval can be understood as a strict, typed, auditable subclass of the broader RAG family — not a rejection of the paradigm.

2.3 Security Ontologies and AppSec Core

Several ontologies have been proposed for security knowledge representation. CWE [11] and CAPEC [12] model the *problem space* (what can go wrong); AppSec Core [7] models the *solution space* (what practitioners do to prevent it). This distinction is relevant for code generation: a developer needs to know what to *do*, not what can go *wrong*. OWASP’s Secure Coding Practices [13] provides an unstructured checklist lacking the typed, weighted structure needed for reproducible retrieval and verification.

AppSec Core [7] provides a normalization ontology for application security: 10 domain slices, 234 typed instances (ControlObjective, Practice, Mechanism, Artifact), with structural invariance across all slices. The companion paper [10] demonstrates that heterogeneous framework requirements (SSDF, ASVS, SLSA) converge on shared objectives within this ontology.

The ontology is supported by a structural index of 4,139 units, each annotated with `document_role`, `normative_weight`, and `heading_path`. This metadata is what makes ontology-grounded retrieval possible — and what plain RAG lacks.

2.4 Model Context Protocol

MCP [8] provides a standardized interface for LLM agents to access external tools and knowledge sources. In this work, MCP is the delivery mechanism, not the contribution. The contribution is the ontological structure of the knowledge being delivered and the retrieval method that produces auditable context.

3. The Problem: Five Missing Properties in Plain Retrieval

This section identifies five properties that security-critical code generation requires and that plain vector-similarity RAG does not provide. The critique is scoped to retrieval without structured metadata.

3.1 No First-Class Provenance

Vector-similarity RAG retrieves text chunks ranked by similarity. The LLM receives content but not the identity of the requirement it represents. Post-hoc, there is no structured mechanism to determine which specific requirement influenced the generated code. For security compliance, where audit trails require traceable links between requirements and implementations, this is insufficient.

3.2 No Epistemic Weight

Security knowledge is heterogeneous: normative requirements (“input **MUST** be validated by allowlist”), operational practices (“use JSON Schema at the API gateway”), implementation mechanisms (“pydantic BaseModel with `strict=True`”), and anti-patterns (“**NEVER** use `eval()` on user input”) carry different epistemic authority. Plain RAG treats all chunks as equal, with no mechanism to distinguish mandatory requirements from informational examples.

3.3 No Reproducible Grounding

The same prompt may retrieve different chunks depending on the embedding model version, index state, or similarity threshold. The security grounding of code generation is therefore non-reproducible: two runs may produce code grounded in different requirement subsets. For functional code this is acceptable; for security, where the question is “were all applicable requirements considered?”, it undermines the audit trail.

3.4 No Completeness Guarantee

Vector similarity retrieves the *most similar* chunks, not *all relevant* ones. If a knowledge base contains seven input validation requirements (VAL-001→007), RAG may retrieve three or four, missing the rest. There is no mechanism to verify that the full requirement family was considered.

This retrieval incompleteness is structurally analogous to the *claim gap* phenomenon in compliance knowledge management [10]: in both cases, a partial surface is mistaken for the applicable knowledge base.

3.5 No Verification Against Grounding

Plain RAG provides no way to verify generated code against the same requirements that grounded it. The retrieved chunks are consumed by the LLM and discarded; there is no structured record of what was provided, and no mechanism to check whether the output satisfies it.

4. Method: Ontology-Grounded Retrieval

4.1 Overview

Ontology-grounded retrieval replaces the embed-search-inject pipeline with a five-stage structured query pipeline:

1. Intent parsing: the prompt is analyzed to identify relevant ontology slices
2. Structured retrieval: the knowledge graph is queried with slice, risk level, and document role filters, returning typed units — not similarity-ranked chunks
3. Context assembly: retrieved units are ordered by epistemic weight (strong requirements first, then practices, then mechanisms), each with an explicit provenance header
4. Grounded generation: the LLM generates code with instructions to cite which requirements it satisfies
5. Verification: the same structured index is queried to verify the output

The method is general: it requires (a) a typed ontology with explicit relations, (b) a structural index with role and weight annotations, and (c) a delivery protocol. We instantiate it with AppSec Core [7] and MCP [8], but the method is not bound to these components.

4.2 Six Distinguishing Properties

Table 1. Property comparison: plain vector-similarity RAG vs. ontology-grounded retrieval.

Property	Vector-similarity RAG	Ontology-grounded
Provenance	Implicit (chunk ID, opaque)	Explicit and first-class (document_role + heading_path)

Property	Vector-similarity RAG	Ontology-grounded
Typing	Absent or weak; all chunks treated equivalently	Explicit: ControlObjective, Practice, Mechanism, Artifact, AntiPattern
Weighting	Absent; no epistemic differentiation	First-class: <code>normative_weight</code> (strong / medium / low)
Reproducibility	Non-reproducible (embedding-dependent)	Reproducible given fixed classification and index version
Completeness	Not assessable (similarity $\neq$ coverage)	Assessable via retrieval contract invariants
Verifiability	Not against grounding (no structured record)	Same structured index for generation and verification

These properties are not binary (RAG has none / ontology has all). They represent a spectrum; Table 1 describes the *baseline* forms of each approach.

4.3 Reproducible Grounding

The retrieval path has two stages with different reproducibility properties:

```
prompt → [classification] → slice set    (probabilistic – intent parser)
slice set → structured query → units    (reproducible – same query, same result)
```

We distinguish classification uncertainty (mapping a prompt to ontology slices — non-deterministic) from retrieval reproducibility (querying the structural index with fixed filters — fully reproducible). Given the same classification, ontology version, and index version, the same requirements are retrieved.

We use the term reproducible grounding rather than “deterministic.” The retrieval is reproducible *conditioned on a fixed classification*. The classification step introduces uncertainty analyzed in Section 7.2.

4.4 Typed Context Delivery

Each unit delivered to the LLM carries its ontological type:

- ControlObjective (strong): non-negotiable requirements
- Practice (strong): operational approaches implementing objectives
- Mechanism (medium): technical implementation patterns (LLM may choose alternatives)
- Artifact (medium): evidence requirements (output should be testable)
- AntiPattern (medium): explicit prohibitions

The following structured context envelope illustrates what the LLM receives for one requirement:

```
slice: ACO-IVF
risk_level: L2
requirement:
  id: VAL-003
  type: ControlObjective
  weight: strong
  description: "Schema validation before deserialization"
  validation_method: "Verify typed model or JSON Schema at API entry point"
  expected_evidence: "Test suite with malformed payload cases"
  provenance:
    document_role: requirements_catalog
```

```
normative_weight: strong
heading_path: "Validação de Requisitos > VAL-003"
```

This envelope enables the LLM to: (a) treat the requirement as non-negotiable; (b) cite it in output (`// VAL-003`); (c) know that evidence (test suite) should accompany the implementation. The same envelope is available to the verification loop.

4.5 The Retrieval Contract

We formalize the retrieval as a contract:

Definition (Retrieval Contract).

Input: prompt p , risk level $l \in \{L1, L2, L3\}$, optional explicit slice hints H

Output: retrieved requirement set $R = \{(id, type, weight, provenance)\}$

Completeness invariant (conditional): for every activated slice s and every ControlObjective $co \in s$ where $co.risk_level \leq l$: $co \in R$. All mandatory requirements for activated slices at the requested risk level must be returned. This invariant is conditional on correct slice activation; completeness is guaranteed with respect to the activated ontology region, not with respect to the user prompt in an unrestricted sense.

Provenance invariant: every element of R carries a (document_role, normative_weight, heading_path) triple traceable to the structural index.

The completeness invariant is testable: given a slice activation and risk level, one can enumerate expected ControlObjectives and verify the retrieval returned all of them. This is not possible with similarity-ranked retrieval, where “enough” is undefined.

4.6 Proportional Application

Risk level is a retrieval parameter, not a post-hoc filter:

- L1 (low risk): L1-mandatory requirements only
- L2 (standard): L1 + L2-mandatory requirements
- L3 (high risk): all requirements including L3-specific and recommended practices

This prevents over-engineering low-risk components and ensures high-risk components receive full treatment.

5. Worked Examples

The following examples are illustrative demonstrations of traceability properties, not comparative evaluations or standalone reproducibility packages. Each traces the intended pipeline from prompt to verification in compact form. Failure modes are discussed in Section 7.

Plain vector-similarity RAG:

prompt $\rightarrow$ embedding $\rightarrow$ similarity search $\rightarrow$ chunks $\rightarrow$ LLM $\rightarrow$ code (no audit trail)

Ontology-grounded retrieval:

prompt $\rightarrow$ slice classification $\rightarrow$ structured query $\rightarrow$ typed requirements
 $\rightarrow$ LLM $\rightarrow$ cited code $\rightarrow$ verification against same index (closed audit trail)

5.1 API Input Validation

Prompt: “Write a Python API endpoint that accepts a JSON payload with user profile data.”

Retrieval: ACO-IVF slice at L2. Returns:

Type	Weight	ID	Requirement
ControlObjective	strong	VAL-001	All input validated by allowlist
ControlObjective	strong	VAL-003	Schema validation before deserialization
ControlObjective	strong	VAL-005	Validation before internal use
ControlObjective	strong	ERR-001	Fail-safe; reject invalid input without revealing internals
Practice	medium	—	Use typed model at API entry point
Mechanism	medium	—	pydantic BaseModel with strict=True
AntiPattern	medium	—	Never use eval() / exec() on user input

Generated code (with requirement citations):

```
from pydantic import BaseModel, ConfigDict

class UserProfile(BaseModel):
    model_config = ConfigDict(strict=True) # VAL-003: schema before deserialization
    name: str
    email: str
    age: int

@app.post("/profile")
async def create_profile(profile: UserProfile):
    # VAL-001: allowlist validation via typed model fields
    # VAL-005: validated before any internal use
    # ERR-001: pydantic returns 422 without exposing internals
    return await save_profile(profile)
```

Verification: VAL-001 ✓, VAL-003 ✓, VAL-005 ✓, ERR-001 ✓, AntiPattern (no eval) ✓. All retrieved requirements addressed. Audit trail complete.

5.2 Kubernetes Container Hardening

Prompt: “Write a Kubernetes deployment manifest for a web application.”

Retrieval: ACO-SPC + ACO-RPR at L2. Returns CFG-001 (no debug), CFG-003 (no hardcoded credentials), securityContext practices, and privileged: true anti-pattern.

Generated manifest includes: runAsNonRoot: true, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, credentials from secretKeyRef, no debug ports exposed. Each security property traceable to a retrieved requirement.

5.3 CI/CD Pipeline Security

Prompt: “Write a GitHub Actions workflow for building and deploying a container image.”

Retrieval: ACO-SCBI + ACO-RPR at L2. Returns provenance attestation, artifact signing, runner isolation, secret injection, and branch protection requirements.

Generated workflow includes: attestation generation, cosign signing, `secrets.*` only, image reference by digest, and approval gate for production. Each step traceable to retrieved requirements.

6. The Verification Loop

The verification loop closes the audit trail. We organize verification into three classes by automation level.

6.1 Syntactic Verification (Fully Automatable)

Checks on presence or absence of concrete constructs: anti-pattern absence (`eval()`, `privileged: true`), configuration presence (`runAsNonRoot: true`), citation validity (does `// VAL-003` exist in the knowledge graph and in the retrieved set?). These are deterministic.

6.2 Semantic Approximation (Semi-Automatable)

Checks requiring limited static analysis: data flow (does user input pass through validation before a database query?), schema enforcement (is strict mode enabled?), citation plausibility (does code near `// VAL-003` actually perform schema validation?). Approximable with SAST tools configured from the same requirement families.

6.3 Human Judgment Required

Checks requiring human review: architectural adequacy (least privilege?), business logic constraints (appropriate fail-safe?), defense completeness (all attack surfaces?). The structured audit trail makes human review tractable but not automatable.

6.4 The Closed Loop

Generation and verification use the same structured index:

- No requirements lost in translation between grounding and verification
- Verification scoped to exactly the relevant requirements
- Audit trail complete: prompt → slice → requirements → code → verification results

This closed loop is not achievable with plain RAG, where retrieved chunks are consumed without structured record.

7. Discussion

7.1 What is Reproducible, What is Not

Ontology-grounded retrieval makes the *grounding* reproducible, not the *output*. Once slice activation is fixed, the same query yields the same requirements. The LLM's generation remains probabilistic. But the security contract is fixed: same requirements, same verification criteria, same audit trail.

7.2 The Intent Parser: Primary Source of Uncertainty

The intent parser — mapping a prompt to ontology slices — is the primary source of uncertainty. If it assigns wrong slices, retrieval is reproducible but *reproducibly wrong*: the completeness invariant holds for activated slices, but the slices themselves may be incorrect.

Mitigation strategies (in order of robustness):

1. Explicit tagging (@AC0-IVF) — bypasses the parser; most reliable, least ergonomic
2. Conservative over-retrieval (top-3 slices) — trades context space for coverage safety
3. Confidence thresholding — low-confidence slices trigger fallback to tagging or over-retrieval

Empirical characterization of parser accuracy is future work.

7.3 Cost and Generalizability

Ontology-grounded retrieval requires a maintained, typed knowledge graph — a non-trivial investment compared to indexing unstructured documentation for RAG. The cost is justified when security verification properties are required. For non-security contexts, standard RAG may suffice.

The method is general in structure: it requires a typed ontology, a stratified index, and a delivery protocol. We instantiate it for AppSec with AppSec Core and MCP. The pattern is in principle applicable to other domains where structured, verifiable grounding is needed — regulatory compliance, medical guidelines, safety-critical engineering — but this broader applicability remains a methodological claim, not a demonstrated result.

7.4 Relationship to Static Analysis

Ontology-grounded retrieval is complementary to SAST:

- Requirement families → SAST rule selection: activating a slice enables the corresponding rule sets
- AntiPatterns → lint rules: each prohibition translates to a static analysis check
- Validation methods → test templates: each requirement’s `validation_method` seeds test generation

Both generation and verification operate from the same structured requirements, ensuring consistency.

7.5 Illustrative Property Snapshot

We report a small illustrative comparison of retrieval properties for the three prompts in Section 5. This is not a controlled evaluation and should not be read as evidence of superiority in code quality or security outcomes; it serves only to make the contract properties operationally concrete. The baseline shown here is a single illustrative instantiation using one embedding model over the same corpus without structured metadata:

Metric	Vector-similarity RAG	Ontology-grounded
Requirements retrieved (of applicable ¹)	3–5 of 7 (varies)	7 of 7 (stable)
Requirements with explicit ID in context	0	7
Epistemic weight attached	No	Yes (strong/medium)
Verification checks executable post-gen	0 (no record)	5–7 (syntactic + semantic)
Audit trail constructible	No	Yes

¹ “Applicable” = all ControlObjectives in activated slices with `risk_level ≤ selected level` (per retrieval contract, Section 4.5).

This snapshot compares *retrieval properties* of a single baseline instantiation, not code quality or security outcomes. A rigorous evaluation with multiple models, prompts, annotators, and statistical controls is future work.

7.6 What This is Not

This method does not claim that LLMs should write security-critical code autonomously. Human review remains essential. What ontology-grounded retrieval provides is structure for that review: reviewers see exactly which requirements grounded the generation and can verify each one.

Nor does it guarantee secure code. The claim is narrower: ontology-grounded retrieval produces auditable, requirement-traceable grounding. The security of the output depends on the requirements' quality, the LLM's adherence, and the verification loop's thoroughness.

8. Limitations

No controlled experiment. The validation is methodological (compact illustrative scenarios making the contract properties concrete), not experimental. A comparative study measuring vulnerability rates and requirement coverage across grounding modes is future work.

Intent parser uncharacterized. The primary source of uncertainty has no accuracy data. Future work should measure classification precision and recall per slice across prompt types.

Single instantiation. The method is demonstrated with AppSec Core and MCP. Generalization to other ontologies and delivery mechanisms is argued but not validated.

Worked examples are happy paths. Failure modes (wrong slice, incomplete retrieval, LLM non-compliance) are discussed but not systematically characterized.

9. Conclusion

We have presented ontology-grounded retrieval as a method for auditable LLM-assisted secure code generation. The method replaces probabilistic vector similarity with reproducible structured queries against a typed security knowledge graph, providing six properties that plain RAG does not inherently offer: first-class provenance, explicit typing, epistemic weighting, reproducible grounding, completeness assessment, and verifiability against the same index.

The retrieval contract formalizes completeness and provenance invariants that make grounding testable. The three-class verification taxonomy makes automation boundaries explicit. The closed verification loop — where the same index grounds generation and verifies output — closes the audit trail.

The method does not make LLM-generated code secure. It makes the grounding auditable: requirements are explicit, provenance is traceable, verification is reproducible. The method is general in structure but validated here only in an AppSec-specific instantiation; its broader applicability remains a methodological claim to be tested empirically in future work.

In a landscape where LLMs generate increasing volumes of code, the ability to say “this code was grounded in these specific requirements, and here is the evidence” is a meaningful step toward responsible adoption.

References

[1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions,” in *Proc. 43rd IEEE S&P*, 2022, pp. 754–768, doi:

10.1109/SP46214.2022.9833571.

[2] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, and L. C. Cordeiro, “How secure is AI-generated code: A large-scale comparison of large language models,” *Empirical Softw. Eng.*, vol. 30, art. 47, 2025, doi: 10.1007/s10664-024-10590-1.

[3] C. Tony, M. Mutas, N. Díaz Ferreyra, and R. Scandariato, “LLMSEval: A dataset for security evaluation of LLM code suggestions,” in *Proc. MSR*, 2023, pp. 588–592, doi: 10.1109/MSR59073.2023.00084.

[4] C. Tony, N. E. Díaz Ferreyra, M. Mutas, S. Dhif, and R. Scandariato, “Prompting techniques for secure code generation: A systematic investigation,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 8, art. 225, 2025, doi: 10.1145/3722108.

[5] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[6] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *NeurIPS*, vol. 33, 2020, pp. 9459–9474.

[7] [Companion paper] AppSec Core: A normalized ontology for security requirements across heterogeneous frameworks. 2026.

[8] Anthropic, “Model Context Protocol specification,” 2024. Available: <https://modelcontextprotocol.io>

[9] B. Peng et al., “Graph retrieval-augmented generation: A survey,” *ACM Trans. Inf. Syst.*, 2025, doi: 10.1145/3777378.

[10] [Companion paper] Coverage-preserving compilation of normative and empirical security knowledge. 2026.

[11] MITRE. Common Weakness Enumeration (CWE). Available: <https://cwe.mitre.org>

[12] MITRE. Common Attack Pattern Enumeration and Classification (CAPEC). Available: <https://capec.mitre.org>

[13] OWASP. Secure Coding Practices Quick Reference Guide, 2010.